

Introduction to Embedded Systems

Course Code: 244066

Chapter 6: STM32 Direct Memory Access (DMA) Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Contents

1. Introduction to Direct Memory Access (DMA)

2. Why DMA is important
   - Limitations of CPU-driven transfers
   - Advantages of DMA (speed, efficiency, parallelism)
3. DMA in STM32 Microcontrollers
   - DMA controller in STM32 (overview)
   - Channels, streams, priorities
4. DMA Transfer Modes
   - Memory-to-Memory
   - Peripheral-to-Memory
   - Memory-to-Peripheral
5. DMA Registers (Overview)
   - Control register
   - Address registers
   - Count register
6. Summary

6.1. Introduction to Direct Memory Access (DMA)

Direct Memory Access (DMA)

- Allows hardware components (like I/O devices) to **transfer data directly to or from the main memory (RAM)** without continuous involvement of the CPU.
- In simple terms, **DMA acts like an automatic data mover.**
- It helps the CPU by taking over repetitive data transfer tasks.
- "Data transfer without disturbing the CPU."

6.1. Introduction to Direct Memory Access (DMA)

Basic Working Principle

Normally, when the CPU handles data transfer:

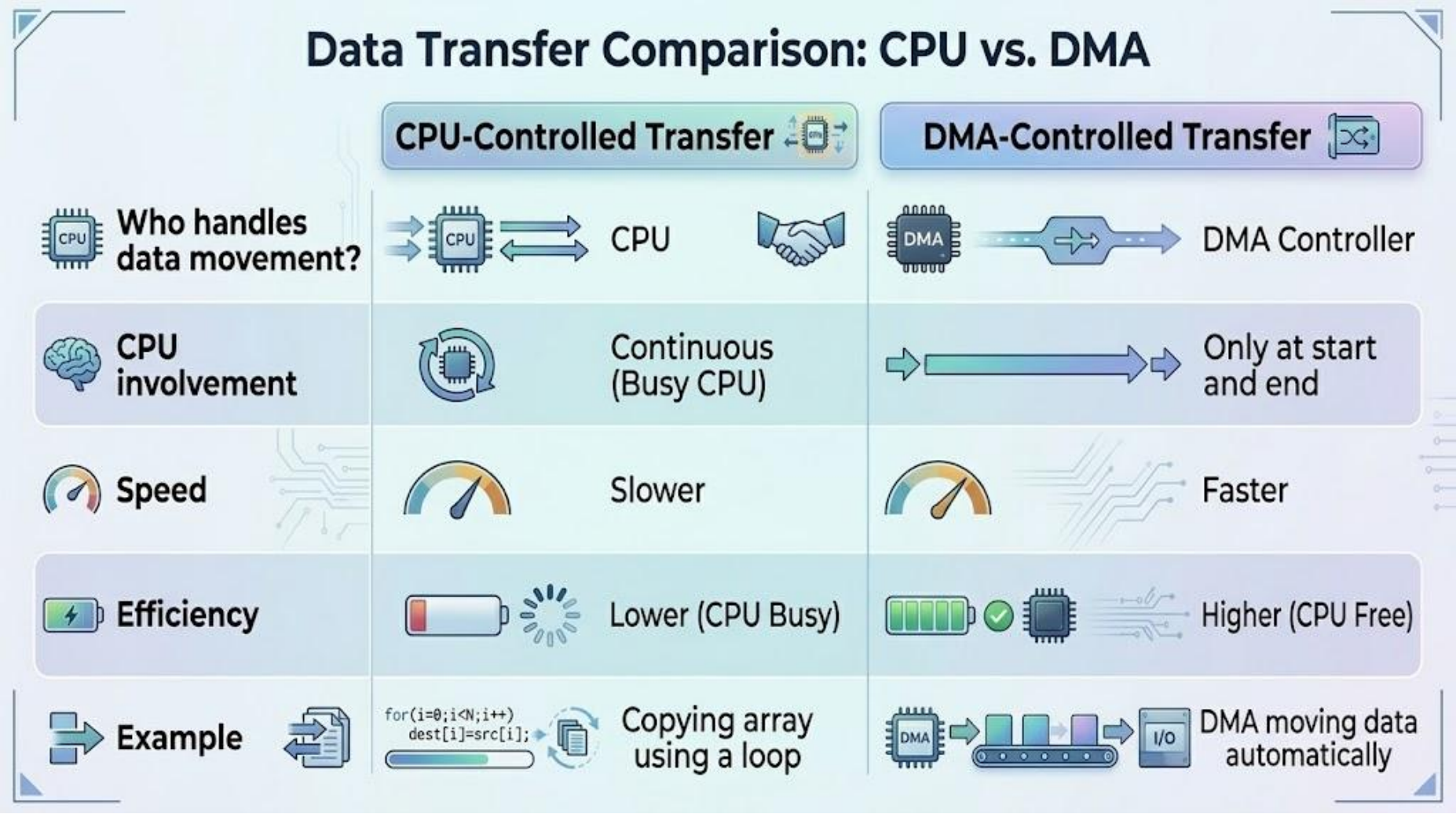
- The CPU reads data from one location (for example, a sensor).
- Then it writes the same data to another location (for example, memory or display).
- This means the CPU must stop its other work during the transfer, which wastes processing time.

6.1. Introduction to Direct Memory Access (DMA)

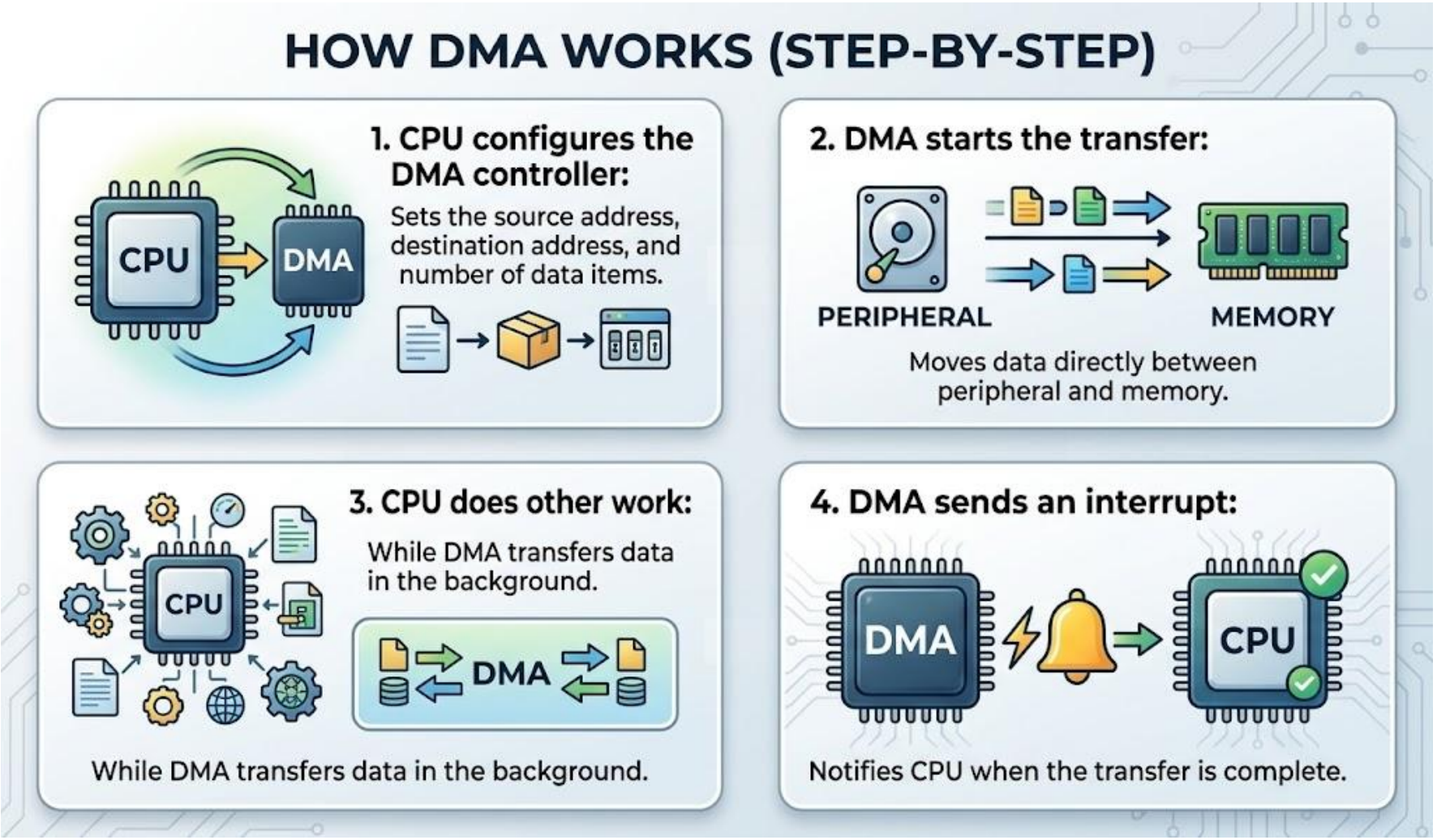
Using DMA:

- The CPU only sets up the DMA controller once (source address, destination address, data size).
- Then the **DMA controller** performs the data transfer automatically.
- The CPU is **free to execute other tasks** in parallel.
- Once the transfer is done, the DMA controller notifies the CPU (using an interrupt).

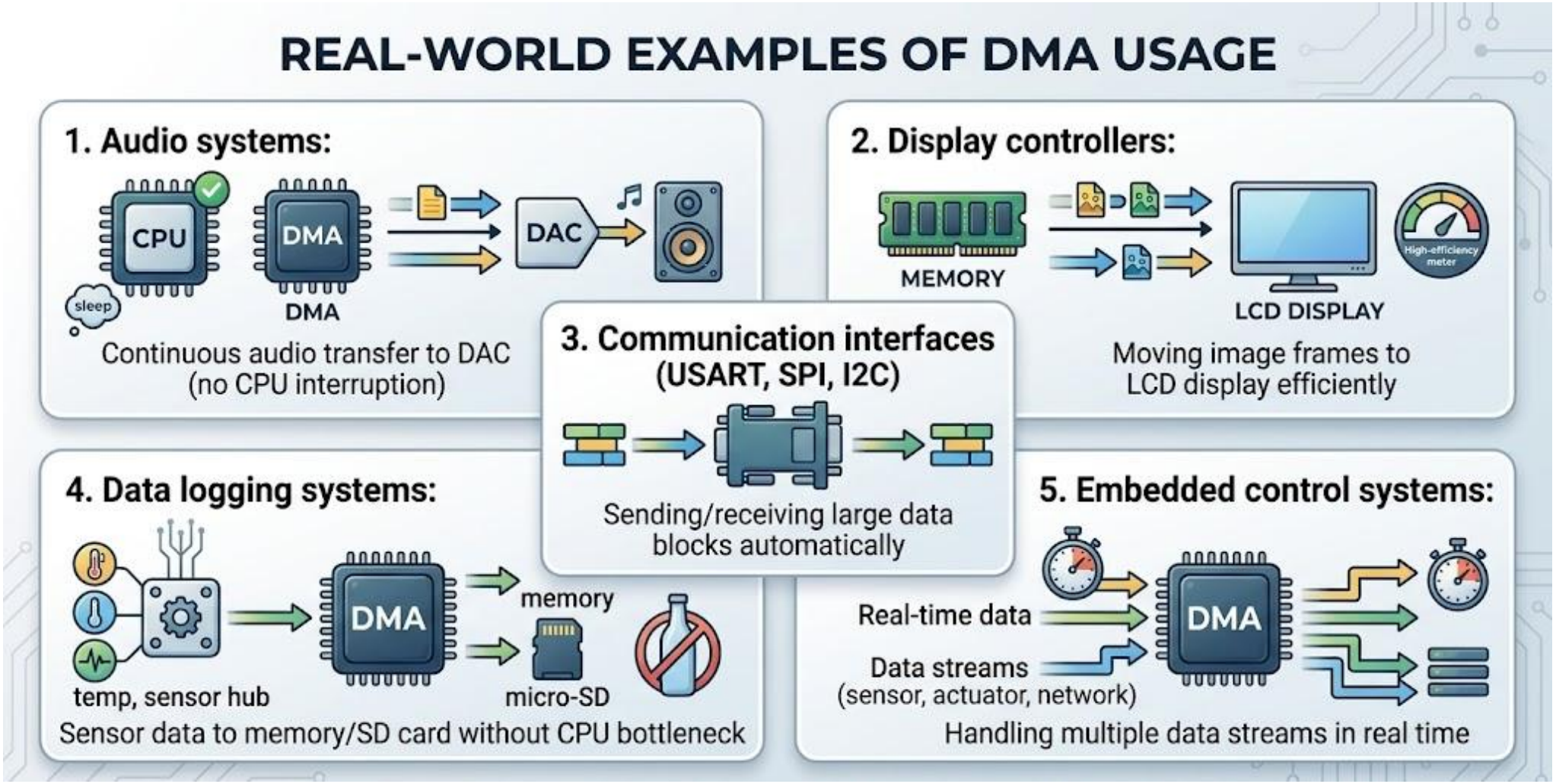
6.1. Introduction to Direct Memory Access (DMA)



6.1. Introduction to Direct Memory Access (DMA)



6.1. Introduction to Direct Memory Access (DMA)



Contents

1. Introduction to Direct Memory Access (DMA)

2. Why DMA is important

- Limitations of CPU-driven transfers
- Advantages of DMA (speed, efficiency, parallelism)

3. DMA in STM32 Microcontrollers

- DMA controller in STM32 (overview)
- Channels, streams, priorities

4. DMA Transfer Modes

- Memory-to-Memory
- Peripheral-to-Memory
- Memory-to-Peripheral

5. DMA Registers (Overview)

- Control register
- Address registers
- Count register

6. Summary

6.2. Why DMA is important

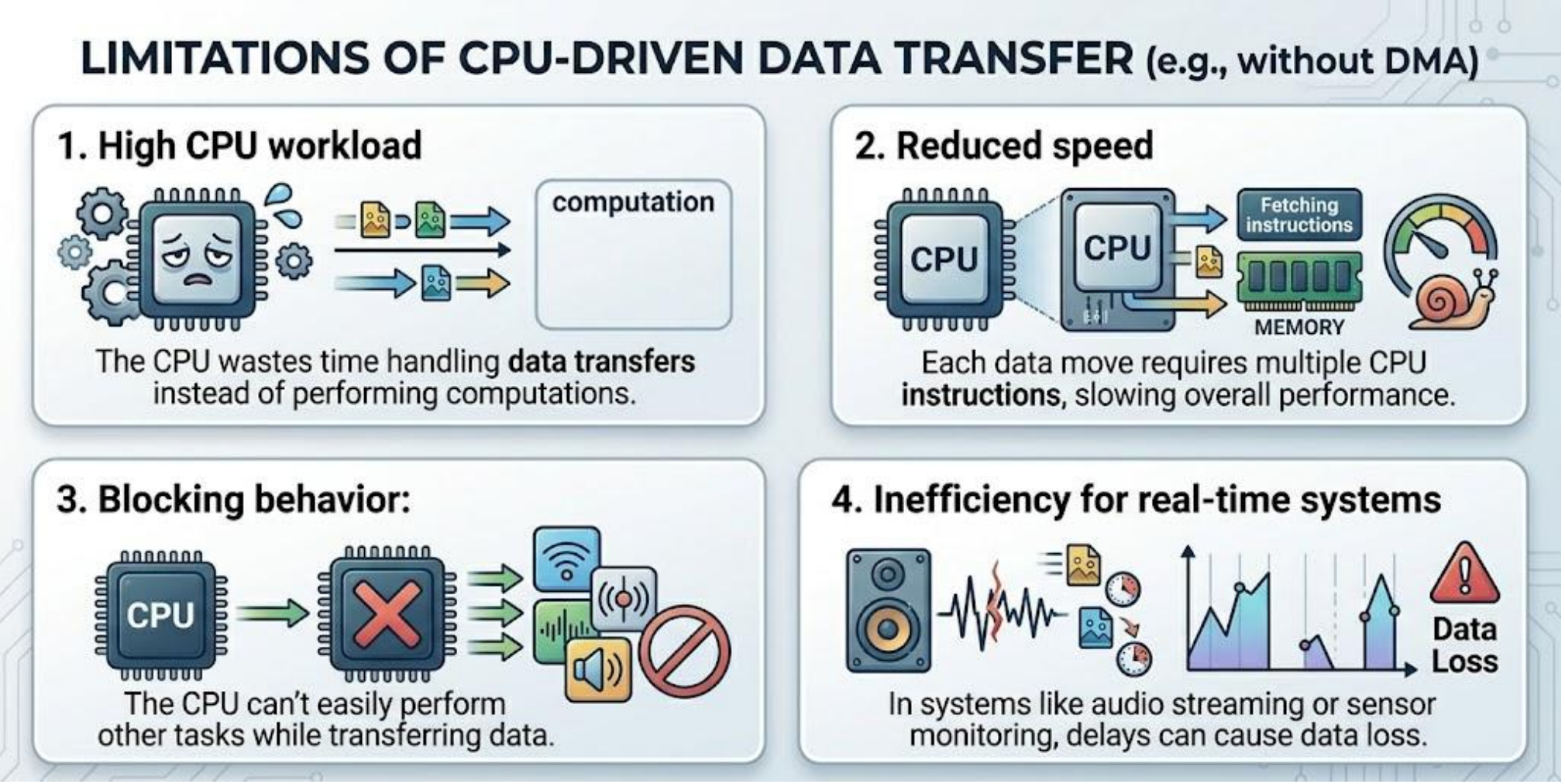
CPU-Driven Transfers

In a typical CPU-controlled transfer (**without DMA**):

- The **CPU** moves data between **peripheral devices** (like sensors, serial ports) and **main memory**.
- For every single byte or word of data, the CPU must:
  - Read the data from the peripheral register.
  - Write the data to memory.
  - Repeat the process until all data is transferred.

This method works fine for **small data**, but has serious limitations for **large or continuous data transfers**.

6.2. Why DMA is important



6.2. Why DMA is important

Example:

- Reading 1000 bytes from a sensor.
- Without DMA, the CPU has to loop 1000 times to copy each byte.
- It can take a huge amount of time and block other tasks.

6.2. Why DMA is important

Advantages of DMA

- **Direct Memory Access (DMA)** was designed to overcome these problems by automating the data transfer process.
- Once configured, DMA moves data between **memory and peripherals** directly, without CPU involvement during the actual transfer.

6.2. Why DMA is important

Key Advantages:

(a) Speed

- DMA transfers are much faster because data moves directly between memory and peripheral.
- CPU instructions are not needed for each byte transfer.
- This allows higher data throughput, especially for continuous data streams (audio, video, serial communication).

6.2. Why DMA is important

(b) Efficiency

- CPU only sets up the DMA once and can then perform other work.
- This means **better utilization of CPU time**.
- The system becomes more responsive because the CPU is free for other important tasks (e.g., user interface, control logic).

6.2. Why DMA is important

(c) Parallelism

- DMA and CPU can work **in parallel**:
  - DMA transfers data in the background.
  - CPU processes other parts of the program.
- This leads to **true multitasking** in embedded systems.

Contents

1. Introduction to Direct Memory Access (DMA)

2. Why DMA is important
   - Limitations of CPU-driven transfers
   - Advantages of DMA (speed, efficiency, parallelism)

3. DMA in STM32 Microcontrollers
   - DMA controller in STM32 (overview)
   - Channels, streams, priorities

4. DMA Transfer Modes
   - Memory-to-Memory
   - Peripheral-to-Memory
   - Memory-to-Peripheral

5. DMA Registers (Overview)
   - Control register
   - Address registers
   - Count register

6. Summary

6.3. DMA in STM32 Microcontrollers

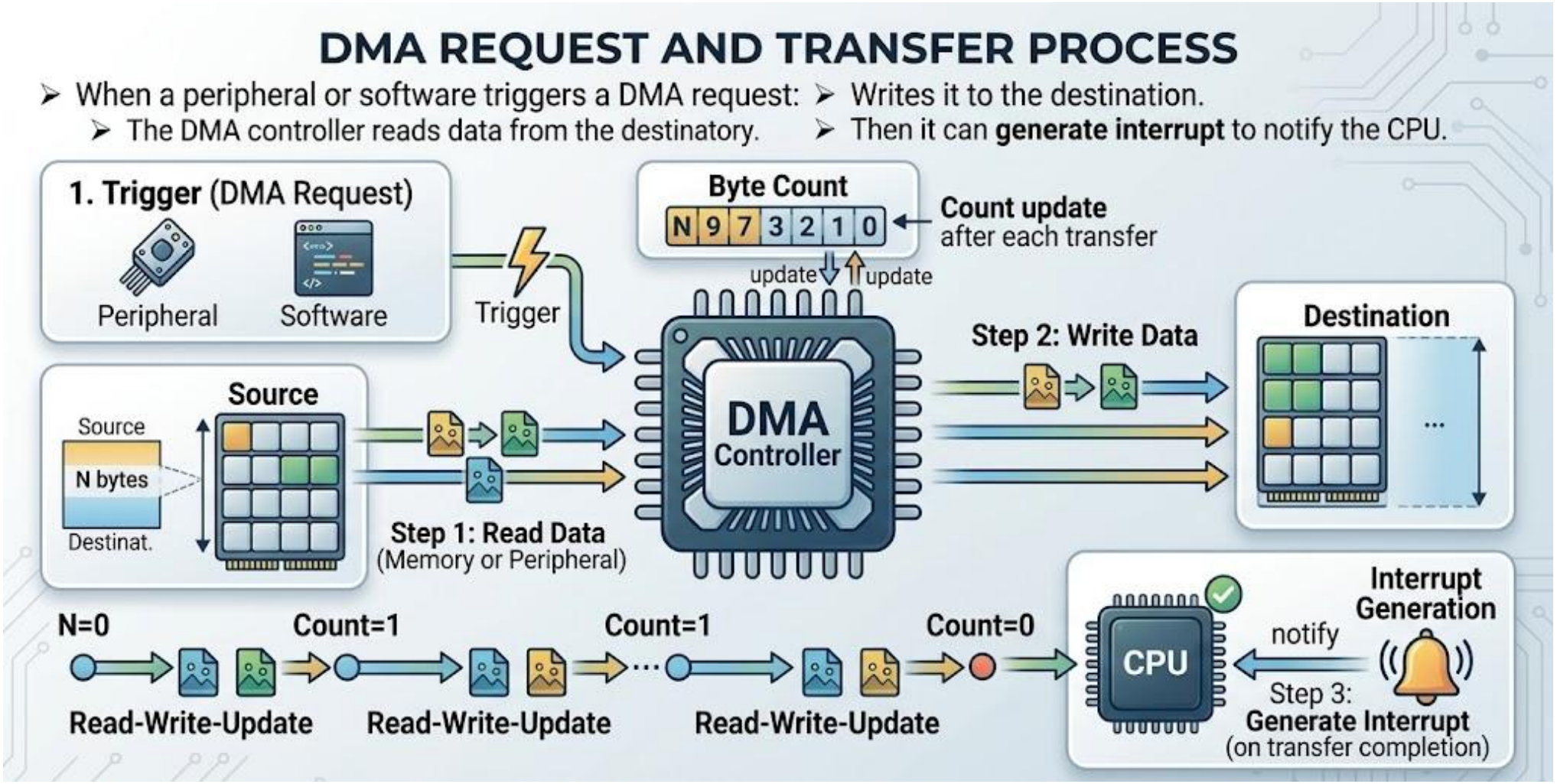
The **DMA (Direct Memory Access) controller** in STM32 microcontrollers is a **dedicated hardware unit** that manages data transfers between **memory and peripherals**, or between **two memory regions**, without needing constant CPU involvement.

6.3. DMA in STM32 Microcontrollers

DMA in STM32 :

- The DMA controller is connected to both:
  - **Memory (SRAM, Flash)**
  - **Peripheral devices** (like ADC, SPI, USART, etc.)
- It uses the **AHB bus** (Advanced High-performance Bus) to access memory quickly.

6.3. DMA in STM32 Microcontrollers



- CPU sets up the transfer only once, DMA takes care of the rest.

6.3. DMA in STM32 Microcontrollers

Example:

- ADC continuously samples analog signals and stores results directly into memory via DMA.
- The CPU only processes the data after a complete block is ready.
- This makes STM32 ideal for **real-time embedded applications** such as sensor reading, audio streaming, and communication interfaces.

6.3. DMA in STM32 Microcontrollers

DMA Channels and Streams

- In STM32, the DMA controller is **divided into multiple "streams"** and each stream may have **multiple "channels."**
- This structure allows **parallel data transfers** and flexible assignment of peripherals.

6.3. DMA in STM32 Microcontrollers

DMA Controller Versions

- Depending on the STM32 family:
- **STM32F1 series:** Uses DMA1 and DMA2 controllers (each has multiple **channels**).
- **STM32F4 and newer:** Uses DMA1 and DMA2 controllers (each with multiple **streams**, and each stream has selectable **channels**).

6.3. DMA in STM32 Microcontrollers

What is a Channel?

- A **channel** represents a **specific data request** from a peripheral.
- For example, USART1_TX might use Channel 4, while ADC1 might use Channel 1.
- The DMA controller listens for data transfer requests from these channels.

6.3. DMA in STM32 Microcontrollers

What is a Stream?

- A **stream** is an actual **data transfer path** between a peripheral and memory.
- Each stream can be configured for:
  - **Source** and **destination** addresses
  - Transfer direction (Memory → Peripheral, Peripheral → Memory, Memory → Memory)
  - Transfer **size** and **mode** (normal, circular, burst)

6.3. DMA in STM32 Microcontrollers

- **Channel** = Who needs service (the requester)
- **Stream** = The road used to transfer data

So, multiple channels can share one stream, but only one can use it at a time (selected by configuration).

6.3. DMA in STM32 Microcontrollers

DMA Priorities

- Since several peripherals might want to use DMA at the same time, STM32 allows setting **priority levels** for each stream.
- DMA can handle **multiple streams simultaneously**, but if two transfers overlap, the one with **higher priority** will get bus access first.

6.3. DMA in STM32 Microcontrollers

Priority Levels in STM32:

- Low priority
- Medium priority
- High priority
- Very high priority

6.3. DMA in STM32 Microcontrollers

Example:

- **Suppose:**
  - ADC is capturing analog data continuously (needs fast service)
  - USART is sending debug text (less frequent)
- **You can assign:**
  - ADC DMA → **Very High Priority**
  - USART DMA → **Low Priority**
- This ensures that the ADC's real-time data is not delayed by slower tasks.

Contents

1. Introduction to Direct Memory Access (DMA)
2. Why DMA is important
   - Limitations of CPU-driven transfers
   - Advantages of DMA (speed, efficiency, parallelism)
3. DMA in STM32 Microcontrollers
   - DMA controller in STM32 (overview)
   - Channels, streams, priorities
4. **DMA Transfer Modes**
   - Memory-to-Memory
   - Peripheral-to-Memory
   - Memory-to-Peripheral
5. DMA Registers (Overview)
   - Control register
   - Address registers
   - Count register
6. Summary

6.4. DMA Transfer Modes

- The DMA controller in STM32 can transfer data in **three main ways**, depending on where the data is coming from and where it needs to go.
- Each transfer mode defines **the direction of data flow** between **memory** and **peripherals**.

6.4. DMA Transfer Modes

1. Memory-to-Memory (M2M) Transfer

- In this mode, data is moved **directly from one memory location to another memory location, without CPU involvement**.
- This is very useful when you need to **copy or move large blocks of data** inside the system.

How It Works

- The CPU sets up:
  - Source address (where data starts)
  - Destination address (where to put data)
  - Number of data items
- DMA transfers the data block automatically.
- Once done, it can trigger an interrupt to inform the CPU.

6.4. DMA Transfer Modes

Use Cases

- Copying data between buffers in RAM.
- Initializing large memory areas quickly.
- Rearranging or sorting image or audio data.

Example

- Copies 1024 data items from one array in memory to another, much faster than using a CPU for loop.

```
DMA_M2M_Config(SRC_Buffer, DST_Buffer, 1024);
```

6.4. DMA Transfer Modes

2. Peripheral-to-Memory (P2M) Transfer

- This mode transfers data **from a peripheral device into memory**.
- It's often used when you want to **store incoming data** (from a sensor, ADC, or serial port) directly into RAM.

How It Works

- The peripheral (like ADC or USART) **generates a DMA request** when new data is ready.
- DMA reads that data from the peripheral's data register.
- It writes the data into a specified location in memory.
- The CPU doesn't need to read each sample — it just processes the buffer later.

6.4. DMA Transfer Modes

Use Cases

- **ADC (Analog-to-Digital Converter):** Store sampled sensor values directly into memory.
- **USART RX (Receive):** Store incoming serial data automatically.
- **SPI RX:** Receive a stream of bytes from another device.

Example

- ADC samples analog input every few microseconds.
- DMA automatically transfers each result into an array in RAM.
- CPU only analyzes the array after DMA signals “transfer complete.”

6.4. DMA Transfer Modes

3. Memory-to-Peripheral (M2P) Transfer

- This mode sends data **from memory to a peripheral device**, automatically and continuously.
- It is used when the CPU wants to **transmit or output a data stream**, but doesn't want to waste time writing each byte manually.

How It Works

- CPU fills a buffer in memory with data to send.
- DMA reads from this buffer.
- It writes each value to the peripheral's data register (like USART, SPI, or DAC).
- When the buffer is sent, DMA can trigger an interrupt to let the CPU know.

6.4. DMA Transfer Modes

Use Cases

- **USART TX (Transmit):** Send strings or binary data automatically.
- **SPI TX:** Send sensor commands or display data.
- **DAC (Digital-to-Analog Converter):** Stream audio samples continuously.

Example

- You want to send “Hello World” repeatedly over USART.
- Instead of sending each character in a loop, DMA transmits the whole string automatically while the CPU continues with other tasks.

Contents

1. Introduction to Direct Memory Access (DMA)
2. Why DMA is important
   - Limitations of CPU-driven transfers
   - Advantages of DMA (speed, efficiency, parallelism)
3. DMA in STM32 Microcontrollers
   - DMA controller in STM32 (overview)
   - Channels, streams, priorities
4. DMA Transfer Modes
   - Memory-to-Memory
   - Peripheral-to-Memory
   - Memory-to-Peripheral

5. DMA Registers (Overview)

   - Control register
   - Address registers
   - Count register

6. Summary

6.5. DMA Registers (Overview)

DMA Registers

- Every DMA transfer in STM32 is controlled through a set of **special hardware registers** inside the DMA controller.

  These registers store the **configuration and status information** that the DMA hardware needs to perform data transfers.

- When we configure DMA in a program (either directly or through the HAL library), we're really just **writing specific values into these registers**.

6.5. DMA Registers (Overview)

- **Why Registers Are Important**
  - Registers tell the DMA **what to do**, **where to get the data from**, **where to send it**, and **how much data to move**.
  - So, when the CPU sets up a DMA transfer, it mainly configures:
  - **Control Register** → *How the transfer should happen*
  - **Address Registers** → *Where to move the data*
  - **Count Register** → *How much data to move*

6.5. DMA Registers (Overview)

1. DMA Control Register (DMA_CCR or DMA_SxCR)

- **Purpose**
  - This register defines the **behavior and settings** of the DMA channel or stream.
  - It's like a **command center** that controls all the transfer parameters.

- **Example**
  - If you enable:
  - MINC = 1 → Memory address increases after each byte/word.
  - CIRC = 1 → DMA automatically restarts (good for ADC or continuous UART RX).
  - DIR = 0 → Data moves from peripheral to memory.
- The Control Register tells **how** the DMA transfer should behave.

6.5. DMA Registers (Overview)

2. Address Registers

- DMA needs to know **where to read** data from and **where to write** it to.
- This is done through two important address registers:

(a) Source Address Register (DMA_CPAR or DMA_SxPAR)

- Stores the **starting address of the data source**.
- Usually this is a **peripheral data register** (like ADC data register, USART data register).

(b) Destination Address Register (DMA_CMAR or DMA_SxM0AR)

- Stores the **starting address of the data destination**.
- Usually this is a **memory buffer** (like an array in RAM).

6.5. DMA Registers (Overview)

Direction Example Table

Transfer Mode	Source Register	Destination Register
Peripheral → Memory	Peripheral data register (e.g., ADC1->DR)	Memory array address
Memory → Peripheral	Memory array address	Peripheral data register (e.g., USART1->DR)
Memory → Memory	Source memory address	Destination memory address

6.5. DMA Registers (Overview)

3. Count Register (DMA_CNDTR or DMA_SxNDTR)

- **Purpose**
  - This register defines **how many data items** (bytes, half-words, or words) should be transferred.
  - Think of it as the **loop counter** for the DMA hardware.
- **How It Works**
  - You load this register with the **total number of data items** to transfer.
  - Each time DMA moves one data item, it **automatically decreases** this counter.
  - When it reaches zero:
    - The transfer is complete.
    - The DMA stops (or restarts if circular mode is ON).
    - An interrupt can be generated (if enabled).

6.5. DMA Registers (Overview)

Summary of DMA Registers

Register Type	Example Register Name	Main Role	Simple Description
Control Register	DMA_CCR / DMA_SxCR	Controls transfer settings	Direction, size, mode, priority
Source Address Register	DMA_CPAR / DMA_SxPAR	Holds source address	From where data is read
Destination Address Register	DMA_CMAR / DMA_SxM0AR	Holds destination address	Where data is written
Count Register	DMA_CNDTR / DMA_SxNDTR	Defines number of transfers	How many data items to move

6.5. DMA Registers (Overview)

Analogy: DMA as an Automatic Courier

Component	Analogy	Function
Control Register	Delivery instructions	Defines how to deliver (speed, route, package size)
Source Address Register	Pickup address	Where to collect the data
Destination Address Register	Drop-off address	Where to deliver the data
Count Register	Number of packages	How many data items to send

Contents

1. Introduction to Direct Memory Access (DMA)
2. Why DMA is important
   - Limitations of CPU-driven transfers
   - Advantages of DMA (speed, efficiency, parallelism)
3. DMA in STM32 Microcontrollers
   - DMA controller in STM32 (overview)
   - Channels, streams, priorities
4. DMA Transfer Modes
   - Memory-to-Memory
   - Peripheral-to-Memory
   - Memory-to-Peripheral
5. DMA Registers (Overview)
   - Control register
   - Address registers
   - Count register

6. Summary

6.6. Summary

- **DMA** allows peripherals to access **memory** directly.
- **Reduces** CPU workload and **improves** system performance.
- Commonly used in real-time and high-speed applications (audio, video, communication).

- DMA **improves** system performance, reduces CPU load, and allows simultaneous execution of multiple tasks — making it **essential** for efficient, real-time embedded systems.

- DMA supports **three transfer directions** to handle different tasks.
- It allows **automatic, high-speed, and CPU-free data movement.**
- Knowing which mode to use is essential for designing **efficient embedded systems**.

6.6. Summary

- The **DMA controller** is like an automatic delivery system in STM32.
- **Channels** decide *who* requests the transfer.
- **Streams** decide *how* the data moves.
- **Priorities** ensure that time-critical transfers get processed first.
- **Memory-to-Memory DMA** is like a super-fast "file copier" inside the microcontroller — it moves data blocks in the background while the CPU does other work.
- **Peripheral-to-Memory DMA** is perfect for collecting continuous data from sensors or communication devices — no CPU interruptions, no missed samples.
- **Memory-to-Peripheral DMA** lets you send large data blocks efficiently, improving speed and freeing the CPU from repetitive transmission loops.

